Reducing the Computational Cost of the 30-Parameter OTI Sensitivity Library: Seven Optimizations

From 2.883X to 1.939X extra cost, full-scale

Ti-6Al-4V single-track laser scan, MFEM/OTI30 sensitivity driver

August 2026

1 Context and Goal

The project computes first-order thermal sensitivities (derivatives of temperature with respect to material, boundary, and laser-source parameters) for a Ti-6Al-4V single-track additive-manufacturing thermal model, using a self-contained multi-directional dual-number type ("OTI30") to propagate all wired parameter directions in a single forward pass. The cost metric used throughout is the **extra-cost multiplier**

$$X = \frac{\text{extra CPU time (sensitivity pass only)}}{\text{nominal CPU time (real temperature solve only)}},$$

matching both this project's own established convention and the reference paper's (Rincon-Tabares et al., *Progress in Additive Manufacturing*, 2025) reporting of a 1.78X extra-cost figure for an equivalent 30-parameter HYPAD-FEM computation.

At the start of this optimization effort, the 30-basis library's full-simulation (8447 accepted time steps, 19,716 DOFs, N=16 wired directions) extra cost was **2.883X**. Seven cumulative fixes, described below, brought this down to **1.939X** – a 32.8% total reduction, and the first result in this effort to cross below the 2X target. Table 1 summarizes the full progression.

State	Nominal [s]	Extra [s]	X
Original (31-basis, single-RHS PARDISO)	1163.35	3353.60	2.883
+ Fix 1–2 (compact-basis + batched multi-RHS)	1167.00	2750.74	2.357
+ Fix 1–5 (+ boundary/domain assembly fixes)	1161.57†	2802.41†	2.411†
+ Fix 1–7 (all fixes, final)	1161.57	2252.69	**1.939**

Table 1: Full-simulation (8447-step) extra-cost progression. †Fixes 3–5 alone produced a small, temporarily higher intermediate reading before fixes 6–7 delivered the larger reduction; see §3.5–§3.7.

2 Benchmark Setup

All numbers below are measured on the `fastverify` mesh (17,160 hexahedral elements, 19,716 DOFs, linear H1 elements), $N = 16$ wired sensitivity directions (of the 30 declared in the target

parameter scheme), MKL PARDISO as the linear solver, −eps 0 (radiation neglected, matching the reference paper's stated assumption). Two scales are used throughout the optimization work: a **500-step short test** (fast iteration, used to validate each fix's correctness and get an initial cost estimate) and the **full 8447-step simulation** (355 reassemblies, the trustworthy number for final reporting – short tests were repeatedly found to *underestimate* the full-scale extra cost, since sensitivities start near-zero and spread through the mesh over time).

A methodological lesson that shaped this work

`callgrind` instruction-count profiling was used early on to guess where computational cost was concentrated. It was later found, via direct wall-clock measurement, to have been **wrong not just in magnitude but in which code region dominated** (see §4). Every fix below was therefore verified two ways before being trusted: (1) a correctness check comparing sensitivity output CSVs against a known-good reference (bit-identical, modulo floating-point noise-floor differences of no physical significance), and (2) a direct `chrono` wall-clock timing measurement – never a proxy metric alone.

3 The Seven Fixes

3.1 Fix 1 – Compact Basis Renumbering

Problem. The OTI30 dual-number type reserved one array slot per *declared* physical parameter (30 total, sparsely numbered 1–30), even though only 16 directions were actually wired. Every arithmetic operation therefore looped over a 31-slot array (30 parameters + 1 reserved for an internal Newton-tangent trick) regardless of how many directions were in use.

Fix. Renumbered the wired directions to a compact, sequential 1–16 range (with slot 17 reserved for the Newton-tangent trick), shrinking `OTI30_NUM_BASES` from 31 to 17. An isolated microbenchmark of the same arithmetic pattern predicted a 2.27× speedup from this change alone.

Result. Real, but far more modest than the microbenchmark predicted: only a ∼3.6% extra-cost reduction. A dead-code block using the old sparse numbering was also found and removed during this work (see §5).

3.2 Fix 2 – Batched Multi-RHS PARDISO Solve

Problem. Although the driver already reused one PARDISO symbolic/numerical factorization per time step across all 16 sensitivity directions, it still issued 16 *separate* PARDISO solve (phase-33) calls per step – one per direction – each re-streaming the resident factor from memory independently.

Fix. Added a batched `SolveMulti()` path to the PARDISO wrapper: all 16 right-hand sides are packed into one column-major buffer and solved in a *single* phase-33 call per step, letting PARDISO reuse the resident factor across all directions in one pass. This matches the architecture explicitly described by the reference paper: "solving all sensitivities of each time increment in a single linear step with a single matrix factorization operation."

Result. The single largest win of the entire effort: `solve_s` dropped from 43.18 s to 12.88 s in the 500-step test (**−70%**). Combined with Fix 1, full-scale extra cost dropped from 2.883X to **2.357X** – an 18.2% reduction, verified at full scale, not extrapolated from a short test.

3.3 Fix 3 – Boundary Quadrature-Point Geometry Cache

Problem. The domain (volume) assembly loop already cached per-element, per-quadrature-point shape functions and Jacobian data (`geom_cache`, built once before the time loop) – but the boundary assembly loop recomputed `CalcShape/SetIntPoint/ Trans.Weight()` from scratch at every quadrature point, every time step. This was initially believed, based on `callgrind` profiling, to be the dominant cost of the boundary loop.

Fix. Added an analogous `bgeom_cache` for boundary faces, built once, reused every step.

Result. Only a small win (−1.6% on `tan_s`) – direct per-line profiling later revealed the geometry recompute itself was only ∼1.8% of the boundary loop's cost, so this fix, while correct, was never targeting the dominant cost (see §4).

3.4 Fix 4 – Skip Radiation Term When Emissivity Is Zero

Problem. The boundary loop unconditionally computed a radiation-coefficient OTI30 arithmetic chain (`hrad_q`, three chained multiplications plus the interpolation feeding it) at every quadrature point, even though every benchmark run uses emissivity = 0 (matching the paper's own "radiation neglected" assumption), which makes this term *identically* zero – in both value and every derivative component.

Fix. Restructured the loop to skip the interpolation and `hrad_q` arithmetic entirely when `emissivity` ≤ 0 (a runtime check, not an approximation – mathematically exact for this case).

Result. Another small win (−0.9% further on `tan_s`), for the same underlying reason as Fix 3: this was not the dominant cost.

3.5 Fix 5 – Domain Interpolation Loop-Interchange

Problem. A decisive diagnostic (direct wall-clock splitting of `tan_s` into domain vs. boundary portions, see §4) revealed that **domain assembly, not boundary, is 99.4% of the tangent-assembly cost** – inverting the earlier `callgrind`-based belief. Within the domain loop, the per-quadrature-point interpolation step accumulated the local temperature *and* all 16 sensitivity fields in one fused loop that touched 16 separate array objects on every iteration, likely blocking compiler auto-vectorization of any single direction's own computation.

Fix. Split the fused loop into the temperature's own simple accumulation plus one clean, contiguous, pointer-based dot-product loop per direction.

Result. A real, modest, same-node-verified win: `tan_dom_s` dropped 4.6% (77.20 s → 73.63 s) in a controlled short-test A/B.

3.6 Fix 6 – Hoisting the Redundant $\dot{g}$ Computation

Problem. A further diagnostic split domain assembly into three sub-costs: interpolation (23%), the material-property (`k_oti`) evaluation (33%), and the final scatter loop (44%, the single largest piece). Inside the scatter loop, a per-node gradient dot product ("`gdot`") was being recomputed inside the per-direction loop even though its value does not depend on which direction is being processed – a *provable* algorithmic redundancy, not a hardware-behavior guess.

Fix. Hoisted the `gdot` computation outside the per-direction loop, computing it once per quadrature point into a small array and reusing it for however many directions survive the loop's own early-exit check.

Result. −40.9% on the scatter sub-cost in the short test, same-node verified.

3.7 Fix 7 – Gating Unnecessary Second-Order Storage Initialization

Problem. Every OTI30 object construction unconditionally zeroed *both* its first-derivative array (`e[]`, always needed) and its second-derivative array (`epp[]`, needed only when second-order sensitivities are requested via `-o2`, which was off for every first-order benchmark run). An earlier, separate attempt this session had gated the *arithmetic operators*' `epp` computation on this flag but never gated the *constructor*'s own initialization – explaining why that earlier attempt measured almost no effect.

Fix. Gated the constructor's `epp.fill(0.0)` call on the same second-order flag.

Result. −27.9% on the `k_oti` sub-cost in the short test, same-node verified.

Combined effect of Fixes 6–7

Together, Fixes 6 and 7 produced the largest single verified improvement of the entire post-Fix-2 effort: `tan_dom_s` dropped **21.3%** (81.00 s → 63.72 s) in a controlled, same-compute-node short-test A/B (both runs confirmed via `sacct` to have executed on identical hardware). At full scale, the improvement was *larger* than this short-test figure predicted (−27.9% vs. −21.3%) – consistent with the mechanism by which Fix 6 works: its benefit scales with how many of the 16 directions survive the loop's early-exit check, and that surviving fraction grows over the course of the simulation (the same reason tangent-assembly cost grows super-linearly with step count). This is the one instance in this effort where a quantitative prediction, made explicitly before the full-scale result was known, was confirmed rather than merely directionally right.

4 Supporting Diagnostics

4.1 The callgrind pitfall

Whole-program `callgrind` instruction-count (Ir) profiling suggested the boundary loop accounted for ∼79% of tangent-assembly cost. This was later shown, via direct `chrono` wall-clock instrumentation, to be backwards: domain assembly is 99.4% of the real cost, boundary is 0.6%. The lesson: instruction-count profiling can be unreliable not only in magnitude but in *which code region dominates*, even comparing two plain-C++ regions with no external library calls involved. Every subsequent diagnostic in this effort used direct wall-clock timing instead.

4.2 Domain-loop sub-split

Splitting domain assembly into interpolation / `k_oti` / scatter sub-costs (via a per-element, not per-quadrature-point, set of timers, to avoid the timer-call overhead itself contaminating the measurement) found the relative shares 23% / 33% / 44% respectively – directly motivating Fixes 6 and 7 as higher-value targets than the interpolation step Fix 5 had already addressed.

5 A Caveat on Precision: Node-to-Node Variance

While investigating the production driver's own instrumentation separately, a same-code, purely-additive change (adding a previously missing timer, with no effect on any other computed value) showed one timing category swing by ∼**34%** between two different compute nodes on this shared, non-exclusive cluster. Since every full-scale run in this effort landed on a different node, absolute full-scale X values should be read with this uncertainty in mind. The 1.939X final figure is believed substantially real – not an artifact of node variance – because the underlying Fix 6/7 improvement

was independently confirmed via a same-node short-test A/B *and* the full-scale result matched the direction and rough magnitude that mechanism predicted in advance, a standard not met by any single unverified full-scale delta on its own.

6 Conclusion

Seven fixes, applied cumulatively and each independently verified for correctness (bit-identical sensitivity output, modulo floating-point noise-floor differences) and measured effect (direct wall-clock timing, never a proxy metric alone), reduced the 30-parameter OTI sensitivity library's full-scale extra cost from 2.883X to **1.939X** – a 32.8% total reduction, crossing below the project's 2X target for the first time. The two largest wins (batched multi-RHS PARDISO, and the combined gdot-hoisting/epp-fill-gating pair) shared a common trait: they targeted a *provable* algorithmic or architectural redundancy rather than a plausible-sounding but unverified hardware-behavior hypothesis – of the seven fixes, those grounded in a concrete, checkable mechanism consistently outperformed those grounded only in profiler output or intuition.